

This is my se lab 4. I want you to analyse the files, This is my github link(https://github.com/SETAPESU26/13_minesweeper).help me solve the bugs in the game and also identify the bugs in the codes of every file in order to understand the working of the game and also help me fix it.

Worked for 31s

I found your [SE Lab 4 — Minesweeper GitHub repository](#), and I've identified the assignment requirements. This is a Python terminal-based Minesweeper game, not a graphical game. Your lab is specifically designed to help you reproduce an existing bug, understand the code, fix it, and extend the game with additional features.



SETAPESU26/13_minesweeper · GitHub

We'll approach it in a way that helps you both fix the code and explain every change to your SE lab evaluator.

1. What your lab requires

Task 1 — Fix board boundaries

Ensure that neighbouring-cell traversal never accesses invalid coordinates, especially at corners and edges. Fix the flood-fill reveal logic.

Task 2 — Winning and flags

Toggle flags correctly, protect flagged cells from accidental reveals, and recognise a win only after every non-mine cell has been revealed.

Task 3 — Difficulty modes

Add Easy, Medium, and Hard modes with different board dimensions and mine counts, without adding data files.

Task 4 — Action-level feedback

Give concise feedback for each player reveal action, without printing repeated feedback for every cell visited internally during flood-fill.

These are the four tasks explicitly listed in your repository's README. The assignment also requires testing invalid inputs, mine hits, flag toggling, all difficulty levels, and a complete win.



SETAPESU26/13_minesweeper · GitHub

2. How the project files work together

main.py

Starts the program

game.py

Reads commands, manages turns, and controls game flow

board.py

Stores mines, neighbouring counts, revealed cells, and flags

Game result

Safe reveal, mine hit, or victory

Your repository also contains README.md and requirements.txt. We should inspect these along with all three Python files rather than replacing the project with an unrelated implementation.

3. Bugs identified in your current code

I inspected the contents of board.py, game.py, main.py, requirements.txt, and README.md on GitHub. Here are the concrete issues I found in the current source.



GitHub

+4

board.py — the main source of bugs

Critical

Bug 1: Incorrect boundary checking

Current code, around lines 28–29:

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols: yield nr, nc
```

The problem is that Python uses zero-based indexing. For a board with six rows, valid row indexes are 0, 1, 2, 3, 4, 5 — not 6. The same applies to columns.

Correct code:

```
if self.in_bounds(nr, nc): yield nr, nc
```

This reuses your existing `in_bounds()` method and fixes the off-by-one error.

What happens because of the bug?

- The neighbour iterator can produce coordinates outside the board.
- Mine counting can incorrectly treat invalid coordinates as safe cells.
- Flood-fill can add phantom cells to the revealed set.
- Revealing a cell near an edge can make the game report an incorrect win or produce an invalid board state.

This is the original defect your lab specifically asks you to reproduce and fix.



GitHub

+1

Important

Bug 2: Invalid board configurations are not validated

Your constructor accepts arbitrary dimensions and mine counts, then passes the mine count directly to `random.sample()`.

For example, asking for more mines than available cells causes a `ValueError`. Zero or negative dimensions and invalid mine counts should also be rejected clearly.

Fix: validate that dimensions are positive integers and that the mine count is between zero and the number of available cells.

Important

Bug 3: Flood-fill can reveal a mine accidentally

The `reveal()` method uses a stack to expand zero-adjacent regions. Its current logic adds neighbouring cells to the stack, including flagged cells, although the processing loop subsequently skips flagged cells.

The flag check prevents a flagged mine from being revealed in the current implementation, but the method does not explicitly reject an invalid starting coordinate. This makes the board logic less robust when called directly outside `game.py`.

Fix: validate the starting coordinate in `reveal()` itself and preserve the rule that flagged cells cannot be revealed.

Important

Bug 4: The win condition relies only on a count

Current code:

```
return len(self.revealed) == self.rows * self.cols - self.mine_total
```

This assumes that every revealed cell is safe and that the mine set contains exactly `mine_total` mines. That assumption is fragile if the board state is corrupted or a mine has been revealed.

Fix: explicitly verify that every non-mine cell has been revealed and no mine has been revealed.

4. Bugs and improvements in the other files

File	Finding	Recommended fix
game.py	Flag-toggle result is ignored.	Tell the player whether a flag was placed or removed.
game.py	No difficulty selection.	Add Easy, Medium, and Hard board configurations.
game.py	No feedback after a successful reveal.	Print one message per player action, not per flood-fill iteration.
game.py	A repeated reveal is silently ignored.	Explain that the cell is already revealed or flagged.
main.py	Entry point is correct.	No bug needs fixing; retain the simple startup structure.
requirements.txt	Correctly specifies no third-party dependencies.	Keep it standard-library-only.
README.md	Describes the required features and tests.	Update the completion checklist after implementing and testing the fixes.

The distinction matters: `main.py` and `requirements.txt` are not inherently buggy. The largest confirmed defect is in the neighbour-boundary condition.



GitHub

+3

5. How to reproduce the original bug before fixing it

Your lab requires evidence of the original defect, so do this before editing the code.

1. Open the repository in VS Code.
2. Open the terminal in the project directory.
3. Run `python main.py`.
4. Reveal cells near the bottom or right edge, especially cells that trigger large zero-adjacent regions.
5. Observe whether the game reveals phantom cells or reports an incorrect result.

To reproduce the underlying boundary bug deterministically, run this small test against the original `board.py`:

```
from board import Board
board = Board(rows=3, cols=3,
mines=0)
print(list(board.neighbors(1, 1)))
```

In a correct 3×3 board, the centre cell (1, 1) has exactly eight neighbours, all within rows and columns 0 through 2. The original condition permits coordinates with index 3, so this test exposes the problem directly.

Record a short video of the original broken behaviour for your submission, as required by your README.



GitHub

6. Fix the code — complete corrected files

Make a backup or create a Git branch before changing the starter. The following implementation preserves your existing modular design and command format while implementing all four lab tasks.

File 1: `board.py`

Replace the contents of `board.py` with the following:

```
import random
DEFAULT_ROWS = 6
DEFAULT_COLS = 6
DEFAULT_MINES = 6
class Board:
    def __init__(self, rows=DEFAULT_ROWS, cols=DEFAULT_COLS,
mines=DEFAULT_MINES, ):
        if (not isinstance(rows, int) or
isinstance(rows, bool) or not isinstance(cols, int) or isinstance(cols, bool)
or rows <= 0 or cols <= 0 ):
            raise ValueError("Rows and columns must be
positive integers.")
        if (not isinstance(mines, int) or isinstance(mines, bool))
```

```

or not 0 <= mines < rows * cols ):      raise ValueError(      "Mine count must be
non-negative and smaller than board size."      )      self.rows = rows      self.cols = cols

```

What changed in board.py?

- Boundary checking: neighbors() now uses self.in_bounds(), preventing phantom cells.
- Board validation: invalid dimensions and mine counts are rejected.
- Flood-fill: flagged cells and already revealed cells are skipped.
- Flagging: invalid coordinates and revealed cells cannot be flagged.
- Win detection: the game checks the actual set of safe cells rather than trusting a count.

One important detail: reveal() still returns a Boolean indicating whether a mine was hit, so your existing game.py can continue to call it in the same way.

File 2: game.py

This version adds difficulty selection, flag feedback, invalid-input handling, and one feedback message per reveal command.

```

from board import BoardDIFFICULTIES = { "easy": (5, 5, 4), "medium": (6, 6, 6),
"hard": (9, 9, 15),}class Minesweeper: def __init__(self):      self.board = Board()
self.difficulty = "medium" def display(self, reveal_mines=False):      b = self.board
print("\n  " + " ".join(str(c + 1) for c in range(b.cols)))      for r in range(b.rows):          cells =
[]          for c in range(b.cols):              pos = (r, c)              if reveal_mines and pos in
b.mines:                  ch = "*"              elif pos in b.flags:                  ch = "F"              elif pos not
in b.revealed:                  ch = "#"              elif pos in b.mines:                  ch = "*"              else:

```

What changed in game.py?

1. Difficulty selection: creates the board in memory using the chosen configuration.
2. Flag feedback: tells the player whether a flag was placed or removed.
3. Input validation: rejects malformed commands and coordinates outside the board.
4. Protected flags: the player must remove a flag before revealing that cell.
5. Action-level feedback: one reveal command produces one feedback message, even if it reveals a large region.
6. Game termination: mine hits and successful completion end the game.

The flood-fill loop remains inside board.py, so feedback is handled at the command level in game.py, not inside the cell-processing loop.

7. main.py and requirements.txt

These files do not need functional changes.

Your existing main.py already starts the game correctly:

```
from game import Minesweeperif __name__ == "__main__": Minesweeper().run()
```

Keep requirements.txt as it is. Your project uses only the Python standard library, so no package installation is necessary.



GitHub

+1

8. Test the fixed game

After saving the changes, open the VS Code terminal in the repository folder and run:

```
python main.py
```

Choose a difficulty, then use commands such as:

```
r 1 1
```

```
f 2 2
```

```
f 2 2
```

```
r 0 3
```

```
r abc 2
```

```
q
```

The expected behaviour is:

- r 1 1 reveals a cell or ends the game if it contains a mine.
- f 2 2 places a flag; repeating the command removes it.
- r 0 3 is rejected as an out-of-bounds coordinate.
- r abc 2 is rejected because the coordinates are not numbers.
- q exits the game.

Automated boundary test

You can also create a temporary file named `test_board.py` and run the following tests without adding any dependencies.

```
from board import Board# Test 1: A corner has exactly three neighbours.board =
Board(rows=3, cols=3, mines=0)neighbors = list(board.neighbors(2, 2))assert
len(neighbors) == 3assert all( board.in_bounds(r, c) for r, c in neighbors)# Test 2: The
centre has eight neighbours.assert len(list(board.neighbors(1, 1))) == 8# Test 3:
Revealing a zero-mine board clears it.assert board.reveal((0, 0)) is Falseassert
board.won()# Test 4: Flags cannot be placed on revealed cells.assert
board.toggle_flag((0, 0)) is False# Test 5: Invalid coordinates are rejected.assert
board.reveal((3, 3)) is Falseassert board.toggle_flag((3, 3)) is False# Test 6: Invalid mine
counts are rejected.try: Board(rows=2, cols=2, mines=4)except ValueError: passelse:
raise AssertionError("Invalid mine count was accepted")# Test 7: All difficulty
configurations are valid.
```

Run the tests:

```
python test_board.py
```

Expected output:

All board tests passed!

This is the expected result for the corrected implementation; I have not executed these tests against your local files, so run them in VS Code to confirm.

9. Final submission checklist

Lab 4 progress

0 of 10

Reproduce and record the original boundary bug

Fix and save `board.py`

Implement difficulty selection and action feedback in `game.py`

Run the automated board tests

Test mine hits, flags, repeated reveals and invalid commands

Play Easy, Medium and Hard modes

Verify a complete win ends the game

Record a 10-second video before the fix

Record a 10-second video after the fix

Include the complete ChatGPT conversation link

Your README explicitly requires before-and-after gameplay videos and the complete LLM chat history link. Make sure you retain the original buggy version long enough to capture the first video.



GitHub

One final recommendation: understand the difference between `neighbors()`, `adjacent_mines()`, and `reveal()` before your evaluation. The first finds valid nearby cells, the second counts mines among them, and the third uses those results to expand a safe region. That relationship is the key to explaining the original bug and why the fix works.